Design Reflection: Doodlebot – The Lost Kingdom

Tala Alothaim · Technical exam · Autonomous navigation game and Grade 5 lesson

Part 1: The Game

Starting with the audience. I began by reading the requirements carefully to understand the task and the purpose of the game. Before coding, I planned the interface around the most important factor: the audience. For Grade 5 students it had to be simple, clear and straightforward, so I chose a calm pastel style, large buttons, one large map at a time, and side menus that show only what a child needs at each stage (Build → Explore → Results).

Characters children can connect with. I wanted Doodlebot to have specific traits and a look that children would connect with. I chose a simple robot illustration (found on Google, from Magnific) as a style reference, then used ChatGPT to generate the same character in different emotions. I made a table of where each emotion should be used: curious while exploring before the map is complete, excited when Princess-Bot enters the sensor range, thinking while planning and so on. Doodlebot was rebuilt as one layered drawing so it can blink, look where it drives and talk; Princess-Bot has a different shape, not just a different colour, so children can tell them apart at a glance.

Key technical decisions.

- Exploration uses BFS on **Doodlebot's own map**, so the "nearest unknown area" is measured by path distance, not straight-line distance. The planner (BFS or A*) also uses only that map, never the real maze.
- Ties are always broken in the same order, so the same maze gives the same run. This keeps the classroom experiment **fair and repeatable**.
- I added an optional welcome page and a four-step animated tutorial on autonomous navigation. It replays frames from the real algorithm, so it never shows anything the game does not do, and students can skip it.

Challenges and trade-offs.

- When Princess-Bot was walled in, Doodlebot kept exploring after sensing her, which confused me at first. I kept the rule (explore until no reachable unknown place is left) because an unexplored square could hide a gap, and made Doodlebot explain this instead.
- The chatbot answered "That's outside my maze!" to almost everything, because speech recognition misheard "Doodlebot" and "maze". I added example answers to the prompt and corrected common mishearings.
- A natural OpenAI voice sounds friendlier but adds about a second of delay and a small cost, so the browser voice stays as an automatic backup.
- While checking the requirements, I noticed I had added several optional items without planning to, because they made the game feel complete (random solvable mazes, animated search, A* vs BFS, a results panel).

Part 2: The Curriculum

Focus. Since the lesson is only 30 minutes, my goal was for students to understand how autonomous navigation starts and works: a robot senses, builds its own map, and only then plans. To make sure this idea settles, I built the lesson around one experiment instead of every game feature, and kept BFS vs A* as an extension.

Structure. The warm-up is a "Mystery Map" in which the class acts as Doodlebot's brain and can only see the squares next to it. Students then watch the tutorial and run one fair test in pairs: the same maze with sensor range 1, then 4. Pairs finish faster, communicate more and share their own perspectives. Most children expect a bigger sensor to give a shorter path, but mapping drops from 78 to 29 steps while the path stays 21. Predicting first and then explaining their real numbers is the evidence of learning, recorded on the worksheet and judged with a short rubric.

Guiding the facilitator. I wrote the lesson plan so a facilitator can follow it step by step: sample sentences to say, a check-for-understanding question, expected answers and troubleshooting, so the main idea we want children to understand is clear.

How I Used AI Tools and Checked Their Work

- **Image source and ChatGPT:** the "Cute robot rocking out" vector illustration on Magnific (Freepik), found through Google, was my style reference; ChatGPT generated the emotion variations.
- **Claude:** my coding partner. I described my plans, design and decisions, and it helped implement and refine the code and the lesson materials.
- **Checking:** 11 automated tests (nearest frontier by path distance, shortest paths on 60 random mazes, edge cases). I also played every edge-case maze myself, tested the OpenAI connection and the voice chat in Chrome, and questioned anything that did not feel right, such as the walled-in behaviour, the chatbot refusals and tutorial notes disappearing too fast for slow readers. Finally, I went through the code, lesson plan, slides and worksheet together to make sure they are consistent: the same maze, numbers, steps and words everywhere.